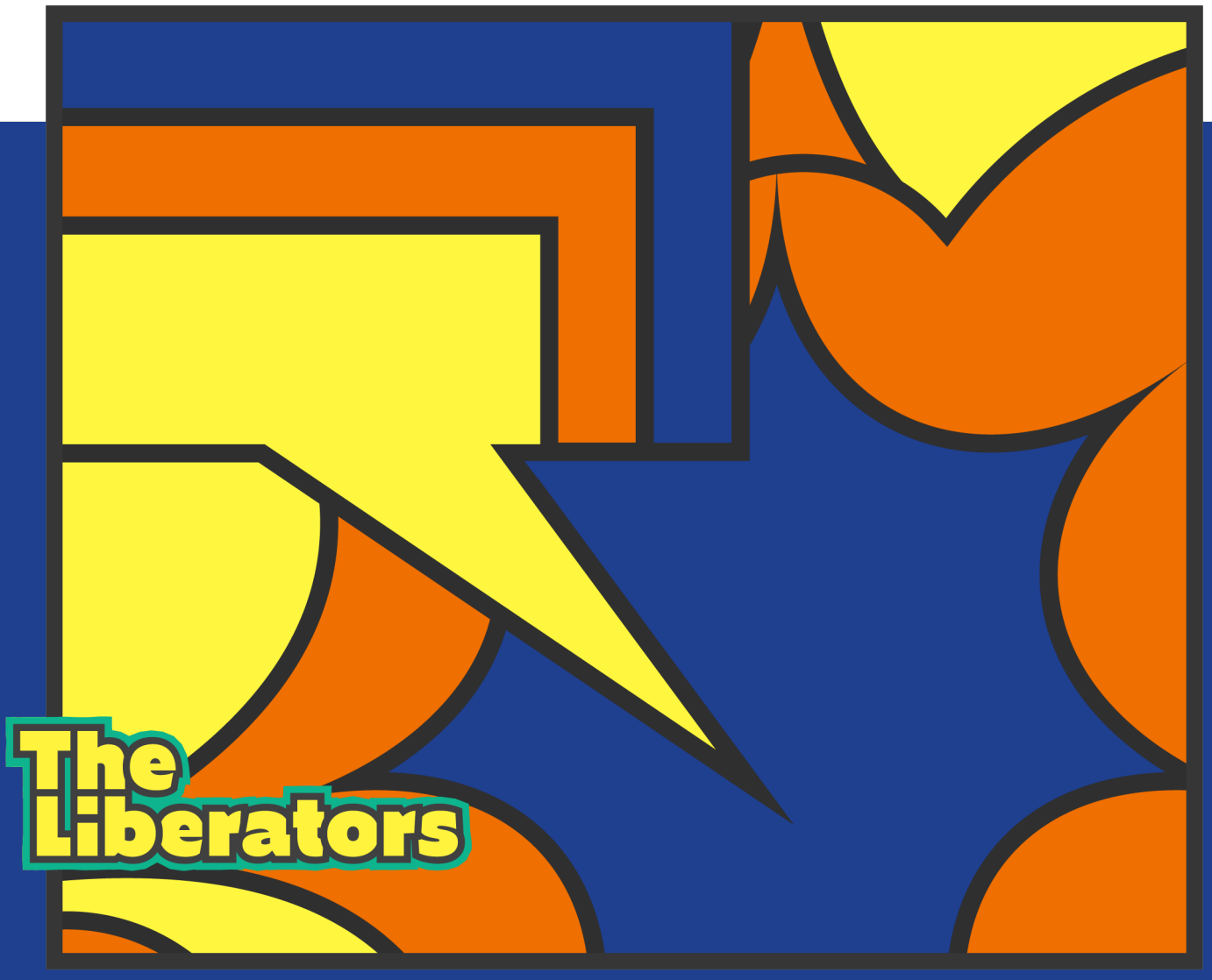
free download

10 Powerful Strategies To Break Down Product Backlog Items

Refine your Product Backlog items into smaller slices of work, and quickly validate assumptions, improve flow, and deliver value to your stakeholders sooner



The Purpose Of This Experiment

Hey there,

If you work with Scrum, you're probably familiar with refinement. It's an activity many teams consider important because it improves flow and reduces the risk of failing the Sprint.

We define 'refinement' as the process of breaking down larger items on the Product Backlog into smaller ones with the purpose of ending with items that can comfortably be delivered to stakeholders in a Sprint. Research shows that teams that spend more time on refinement are also more likely to release frequently. In turn, they have more satisfied stakeholders and higher team morale. Explore our research for more information.

Breaking down large items is a skill that Scrum teams have to learn. While it is often easy to break down work into horizontal slices, across technical boundaries or components, this results in items that cannot be delivered on their own. What good is a new table in the database if the interface doesn't support it yet?

In this paper, Christiaan offers 10 strategies that allow teams to break down work into vertical slices. Each slice can be released and inspected on its own. For each strategy, we include questions that you can ask to help Scrum teams slice in that manner.

Give it a try and let us know how it went!

Barry & Christiaan

The Liberators

Difficulty / Effort

Required Skill	★★★☆☆	Breaking down work can be challenging, work together as a team to make it happen.
Impact On Survival	★★★★★	Small items of work help you validate assumptions quickly, learn what is needed, improve flow, deliver value sooner, and manage risk. So yes, it has a huge impact on survival.

Why And When To Break Down Items

Software development is inherently unpredictable. Some Product Backlog Items (PBI's) on a backlog will require more effort than expected, while other items will require less. So if the work for a sprint contains only a few large items, the impact of underestimating the work on even a single item will have a profound impact on the sprint. And since larger items tend to be harder to estimate and understand, the potential for a failed sprint increases further.

Experienced Scrum teams spend time and effort to break down large PBI's into smaller stories. Although they sometimes do this during the planning of a new Sprint, they have learned that this process of 'refinement' should be continuous in order to make sprints — and particularly sprint planning — run smoother. So when a sprint is underway, the team also spends time breaking down PBI's for the next sprint, and maybe even for the sprint after that. But this is done in a 'just-in-time'-manner, so the team spends increasingly less time on sprints that are increasingly further down the road.

This process of breaking down work improves shared understanding, increases the accuracy of estimation, and facilitates the Product Owner in the prioritization of work. But it isn't easy to do well, and takes practice to do it 'on the fly'. Thankfully, members of the Agile community — like Dean Leffingwell, Richard Lawrence and Mike Cohn — have devised a number of useful strategies for breaking down work into smaller bits.

Why A Vertical Break-down Is Superior To A Horizontal One

Broadly speaking, there are two approaches for breaking up large PBI's. The first approach is called 'horizontal break-down', and involves breaking down stories by the kind of work that needs to be done, or the layers or components that are involved. So, work that has to be done for the UI, the database, some components, a front-end, and testing are split up into technical items on the Backlog. This doesn't work well in Scrum for several reasons:

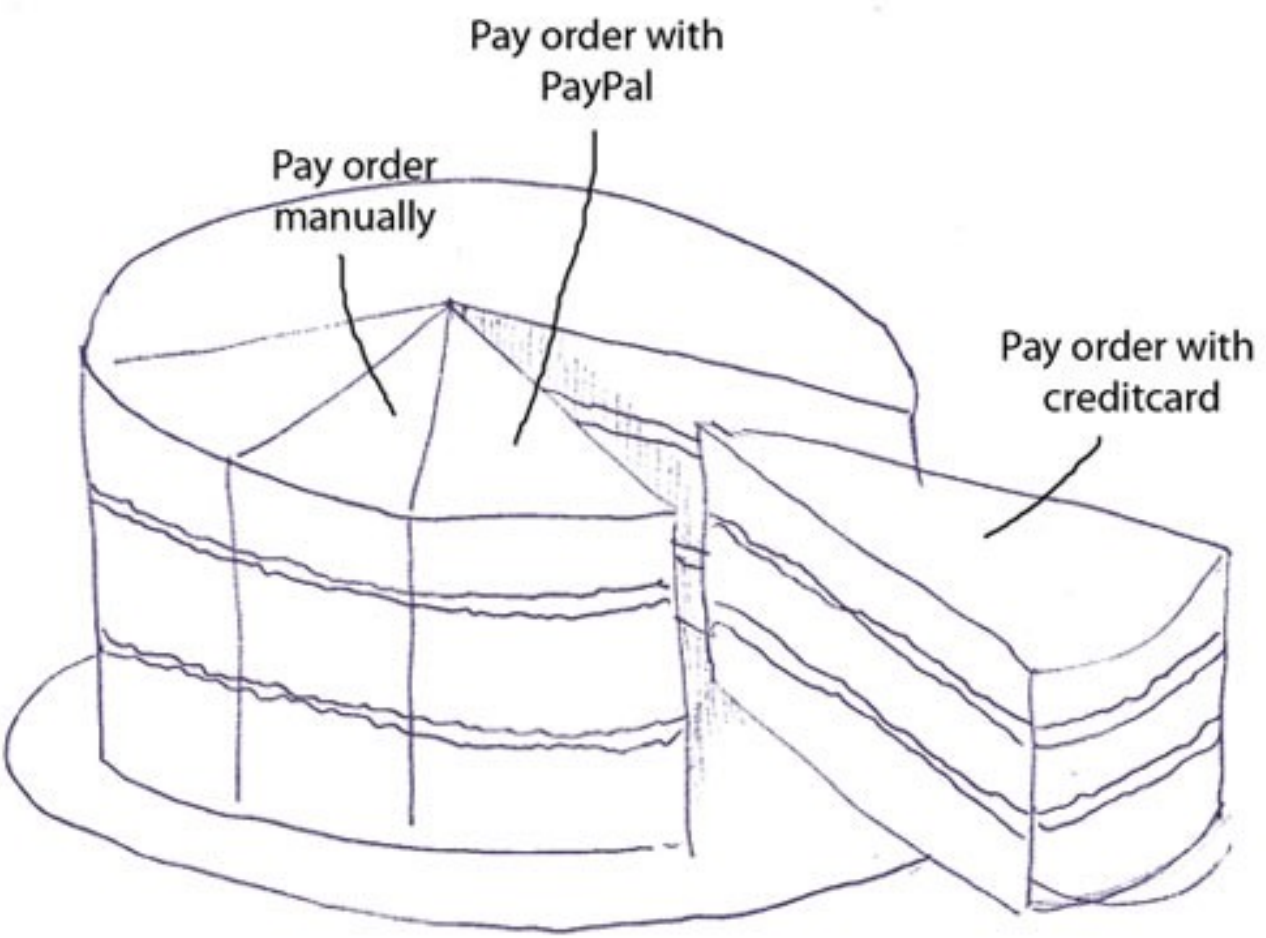
- Individual items do not result in working, demonstrable, software: Suppose that a team works on an order process for a webshop in a sprint. If they would split up the PBI horizontally, they would end up with work for design, database, front-end, and testing. Although the items are certainly smaller, they don't deliver working software on their own. After all, a new bit of functionality can't go live when only the UI is finished, or when only the database was modified. It is also a bad idea to go live without sufficient testing. So, individual items do not result in working, demonstrable, software, and — by extension — business value. Only the combination of all the work generates business value. But only if all the tasks are completed. And this is often a problem, as the next bullet explains;
- Increases bottlenecks, instead of reducing them: Horizontal break-down is often accompanied by 'silo thinking'. Every member is taken from one of the silos required for software development. The 'design guy' will take care of the design, the 'database guy' will set up the database, the 'developer' writes the code and the 'tester' does the testing. If team members are not interchangeable (which is often the case when using this approach) there is a good chance of bottlenecks. If the 'design guy' can't get his work done on time, this will impact the tasks that follow the design. Because team members can't help each other out, every delay, problem, or interruption will impact the entire sprint;
- Horizontal slices can't be prioritized: How can a Product Owner prioritize a backlog if it consists of horizontal slices? Because none of the items deliver business value or working software on their own, a Product Owner will be unable to prioritize work. There is also a good chance that the slices will be fairly technical, which creates distance and misunderstanding between the Product Owner and the team;

Although a horizontal break-down of PBI's will result in smaller items, it severely limits the ability of a team to deliver working software, work around bottlenecks and prioritize work. It, therefore, increases the risk of failing the sprint. A bad idea.

In Scrum, a vertical break-down is more useful, in which PBI's are broken down into smaller PBI's (instead of technical tasks). If PBI's are broken down vertically, they are broken down in such a way that smaller items still result in working, demonstrable, software. The functionality will not be split across technical layers or tasks but across functional layers. So, if the PBI is 'As a customer, I can pay for my order, so I can get my products', it can be broken down into smaller PBI's like 'As a customer, I can pay by wire transfer, so I can get my products' or 'As a customer, I can pay with credit card, so I can get my products'.

A metaphor will further clarify the difference. Imagine slicing a cake with layers of cream, fruit, and cake. If you would cut a cake horizontally, every guest would only get a slice of cake, cream, or fruit. I'm sure that your guests would've expected something else. Getting just a single layer of the cake does not allow them to sample the entire cake. A friendlier approach is to slice the cake up in vertical slices, each with a bit of cake, cream, and fruit.

There are many strategies available for breaking down large PBI's. We have collected ten of the most useful ones. As you will see, these strategies not only help in breaking down PBI's, they also help in deciding what is important and what is not. This way, the Product Owner can more easily prioritize the work and make more informed decisions on what the Scrum team should spend time on.



A Poster With The 10 Strategies

Breaking down Product Backlog Items, a cheatsheet

"Can we break this user story down (vertically) on … "
(and make it simpler, more understandable and easier to estimate & order)

1. Workflow steps? What steps does a user perform? Are all of the steps necessary now? Can steps be simplified for now? *Steps in an order process, like selecting a payment option, delivery method, etc.*	**2. Business rules?** What rules apply to this story? Are all business rules necessary now? Can simpler rules suffice? *Rules in an order process (no order below 10 dollars, no shipping outside US)*
3. Happy / unhappy flow? What does the happy / unhappy flow look like? Are all unhappy flows necessary (right now)? Can unhappy flows be simplified (for now)? *Failures during a webshop order process and possible recovery options*	**4. Input options?** Which platforms are supported? Are all platforms required (right now)? Are some platforms harder to implement than others? *Tablet, iPhone, desktop, touchscreen*
5. Data types and parameters? What datatypes are supported and relevant? Which parameterized views are there? Are all parameters relevant at the moment? *Different search options / strategies or different kinds of reports (tables, graphs, etc.)*	**6. Operations?** What operations does the story entail? Are all operations necessary right now? *Breaking down on CRUD (create, read, update, delete)*
7. Test cases? What test scenarios are used to verify this story? Are all test scenarios relevant at the moment? *Some test scenarios may be very complex, but not highly relevant at this time*	**8. Roles?** Which roles are involved in this story? What can the roles do? Are all roles necessary now? *A customer can create orders, administrators can manage orders, etc.*
9. Browser compatibility? What browsers have to be supported? Are all browsers important at this point? *Ignoring support for Internet Explorer 9 because only a fraction of users makes use of it*	**10. Optimize now vs optimize later?** What optimizations can we think of (UX/UI)? Are all optimizations necessary now? *Implementing autocomplete for addresses, usage of GPS-location*

Created by Christiaan Verwijs | theliberators.com
Inspired by Dean Leffingwell, Richard Lawrence, Mike Cohn and others

The Liberators

Read the full post at
https://bit.ly/tl10breakdownstrategies

free download

Strategy 1: Breaking Down By Workflow Steps

If PBI's involve a workflow of some kind, the item can usually be broken up into individual steps. Take a PBI for an order process of a regular webshop:

"As a customer, I can pay for the goods in my shopping basket, so that I receive my products at home."

Given a fairly standard order procedure, we could identify the following steps:

- As a customer, I can log in with my account, so I don't have to re-enter my personal — information every time;
- As a customer, I can review and confirm my order, so I can correct mistakes before I pay;
- As a customer, I can pay for my order with a wire transfer, so that I can confirm my order;
- As a customer, I can pay for my order with a credit card, so that I can confirm my order;
- As a customer, I receive a confirmation email with my order, so I have proof of my purchase;

By breaking down a large PBI like this, we have improved our understanding of the functionality and our ability to estimate. It will also be easier for a Product Owner to prioritize the work. Some steps in the workflow may not be important right now and can be moved to future sprints. Perhaps the order confirmation email can be done by hand for the time being, or customers can only pay with credit card for now. It might also be ok to require customers to (temporarily) re-enter their personal information for every order. This will certainly limit the functionality of the webshop, but it does allow a Scrum team to review (and maybe even deploy) the completed functionality at the end of the sprint, test it and use the feedback to make necessary changes.

Strategy 2: Breaking Down By Business Rules

PBI's often involve a number of explicit or implicit business rules. Take this PB, again for the order process of a regular webshop:

As a customer, I can purchase the goods in my shopping basket so that I can receive my products at home

Given a fairly standard order procedure, we could identify the following 'business rules':

- As a shop owner, I can decline orders below 10 dollars, because they aren't profitable;
- As a shop owner, I can decline customers from outside the US, because the shipping expenses make these orders unprofitable;
- As a shop owner, I can reserve ordered products from stock for 48 hours, so other customers see a realistic stock;
- As a shop owner, I can automatically cancel orders for which I have not received payment within 48 hours, so I can sell them again to other customers;

Business rules are often implicit, so it will take some skill and analytical prowess to unearth them. It may help to employ another strategy (described below, #7); how is the functionality going to be tested? Often, test cases imply important business rules. Once the business rules have been identified, it will again have improved our understanding and ability to estimate. The product owner may decide that some business rules are not important for now, or can be implemented in a simplified form. Perhaps it's ok for now to manually return unpaid products to stock when a payment has not been received within 48 hours, or orders can be canceled manually. Even more pragmatically, a message on the site explaining that 'orders are not shipped outside the US' or that 'the minimum order price has to be at least 10 dollars' may be sufficient for now. It will save time and money required to enforce this business rule.

Tip: With Small PBI's You Get Work Done More Quickly

Strategy 3: Breaking Down By Happy / Unhappy Flow

Functionality often involves a happy flow and one or more unhappy flows. The happy flow describes how functionality behaves when everything goes well. If there are deviations, exceptions, or other problems, unhappy flows are invoked. Take this PBI for logging in to a secure website:

As a customer, I can log in with my account, so that I can access secured pages

If we consider a regular login procedure, we can identify a happy flow and several potential unhappy flows:

- As a user, I can log in with my account, so that I can access secure pages (happy);
- As a user, I am able to reset my password when my login fails, so I can try to log in again (unhappy);
- As a user, I am given the option to register a new account if my login is not known, so I can gain access to secure pages (unhappy);
- As a site owner, I can block users that log in incorrectly three times in a row, so I can protect the site against hackers (unhappy);

Unhappy flows describe exceptions. By identifying the various flows, we more clearly understand the required functionality. A Product Owner can more easily decide what is important right now. Perhaps blocking users after three failures is not important right now, because there are only a handful of users in the first version anyways. Or perhaps a reset of passwords can be done by sending an email to a customer care employee for the time being. Again, by splitting up functionality we can more accurately ask questions about the business value, and make decisions accordingly.

Strategy 4: Breaking Down By Input Options / Platform

Most web applications have to support various input options and/or platforms, like desktops, tablets, mobile phones, or touchscreens. It may be beneficial to break down large items by their input options. Take a digital Scrum Board for a team:

As a team member, I can view the Scrum Board, so I know how we're doing as a team

We can identify the following input options:

- As a team member, I can view the Scrum Board on my desktop, so I know the status of the sprint;
- As a team member, I can view the Scrum Board on my mobile phone, so I know the status of the sprint;
- As a team member, I can view the Scrum Board on a touchscreen, so I know the status of the sprint;
- As a team member, I can view the Scrum Board on a print-out, so I know the status of the sprint;

By breaking down large items this way, the Product Owner can more easily prioritize which input options or platforms are more important. A desktop version is likely to be sufficient for now, while a mobile version can be pushed to a future sprint. Or perhaps the print-out can be implemented with a simple PDF capture for the time being, without having to create a version specifically suited for print.

Tip: Prevent Big Bang Releases With Small PBI's

Strategy 5: Breaking Down By Data Types Or Parameters

Some PBI's can be split based on the data types they return or the parameters they are supposed to handle. Take, for example, a search function for a webshop:

As a customer, I can search for products so I can view and order them

There are many ways to search for a product. All these potential parameters can be considered and broken up into smaller PBI's:

- As a customer, I can search for a product by its product number, so I can quickly find a product that I already know;
- As a customer, I can search for products in a price range, so that the search results are more relevant;
- As a customer, I can search for products by their color, so that the search results are more relevant;
- As a customer, I can search for products in a product group, so that the search results are more relevant;

By breaking down the search functionality in this manner, we more clearly understand what kind of search parameters will be used. This allows us to more accurately estimate the functionality, but it also allows a Product Owner to make decisions about priority. Maybe paging is not yet relevant because of the small number of products. It might become relevant when the number of products grows. Or maybe some of the search functionality can be implemented in a simplified manner for now. Another example of this strategy is when breaking up functionality involving management information based on the returning data types. Some of the information can be presented visually in charts or graphs (data types), but can also be displayed in a tabular format (datatype) for now. Perhaps the Product Owner is ok with exporting the data to Excel and manually creating all the graphs and charts there for the time being.

Strategy 6: Breaking Down By Operations

PBI's often involves a number of default operations, such as Create, Read, Update, or Deleted (commonly abbreviated as CRUD). CRUD operations are very prevalent when functionality involves the management of entities, such as products, users, or orders:

As a shop owner, I can manage products in my webshop, so I can update price and product information if it is changed

By identifying the individual operations required for this functionality, we can derive the following smaller bits of functionality:

- As a shop owner, I can add new products, so customers can purchase them;
- As a shop owner, I can update existing products, so I can adjust for changes in pricing or product information;
- As a shop owner, I can delete products, so I can remove products that I no longer stock;
- As a shop owner, I can hide products, so they cannot be sold for the time being;

When presented with this strategy, many teams wonder if the smaller items actually deliver business value. After all, what's the point of only creating entities when you cannot update or delete them afterward? But perhaps the Product Owner has such a limited number of products, that editing or deleting can be done in the database directly. Sometimes, an operation can be easily implemented in a simplified form. Deleting a product can be done in two ways; you can completely drop the record and all associated records, or you can 'soft delete' a product. In that case, the product is still in the database, but it is marked as 'deleted'. Sometimes one of these approaches is easier to implement and can be 'good enough' for now.

Tip: Release Early And Often With Small PBI's And Celebrate With Pie

Strategy 7: Breaking Down By Test Scenarios / Test Case

This strategy is useful when it is hard to break down large PBI's based on functionality alone. In that case, it helps to ask how a piece of functionality is going to be tested. Which scenarios have to be checked in order to know if the functionality works? Take a task planning system:

As a manager, I can assign tasks to employees, so they can work on tasks

If we consider this functionality based on potential scenarios, we can break down the item into:

- Test case 1: If an employee is already assigned, he or she cannot be assigned to another task;
- Test case 2: If an employee has reported in sick, he or she should be visually marked;
- Test case 3: If an employee has reported in sick, he or she cannot be assigned to a task;
- Test case 4: If an employee is not yet assigned, they can be assigned to a task;
- Test case 5: Employees can be assigned with a touchscreen monitor;

This strategy actually helps you apply the other strategies implicitly. By thinking about testing, you automatically end up with a number of business rules (#1, #2, #3, and #4), (un)happy flows (#1, #2, and #3), and even input options (#5). Sometimes, test scenarios can be very complicated because of the work involved to set up the tests and work through them. If a test scenario is not very common, to begin with or does not present a high enough risk, a Product Owner can decide to skip the functionality for the time being and focus on test scenarios that deliver more value. In other cases, test scenarios can be simplified to cover the most urgent issues. Either way, relevant test scenarios can be easily translated into backlog items and added to the sprint or the backlog.

Strategy 8: Breaking Down By Roles

PBI's often involves a number of roles (or groups) that perform parts of that functionality. Take a PBI to publish new articles to a public newspaper website:

As a news organization, we can publish new articles on our homepage, so customers have a reason to return periodically

By considering the various roles that are involved, we can break the functionality down into the following bits:

- As a customer, I can read a new article, so I can be informed of important events;
- As a journalist, I can write an article, so it can be read by our customers;
- As an editor, I can review the article before putting it on the site, so that we can prevent typos;
- As an admin, I can remove articles from the site, so that we can pull offending articles;
- As a customer, I can review and confirm my order, so I can correct mistakes before I pay;

By breaking down functionality into the roles that have to perform bits of that functionality, we more clearly understand what functionality is needed and can more accurately estimate the work involved. Writing PBI's is a great way to apply this strategy. It also helps the Product Owner to prioritize. Some roles may be less relevant at the moment, and can be implemented later. Perhaps there is no need at the time for an editor. Or the editor can be called by a journalist to check the article manually.

Tip: Small PBI's Make You More Responsive To Stakeholders' Needs

Strategy 9: Breaking Down By 'Optimize Now' Versus 'Optimize Later'

Often, PBI's can be implemented in varying degrees of perfection and optimization of the functionality described. Take this PBI:

As a visitor, I can search for hotels in a neighborhood, so I can narrow my search

A story like this can be broken down into varying degrees of optimization:

- As a visitor, I can search for hotels based within a radius from an address, so I can narrow down my search;
- As a visitor, I can enter the zip code to auto-complete the address, so I don't have to enter the entire address manually;
- As a visitor, I can use my location (GPS) to search in the neighborhood, so I don't have to enter the address manually;
- As a visitor, I get the most-searched hotels in the neighborhood right away, while other hotels are loading in the background, so I more quickly get results;

Let me be clear on one thing: this strategy is not supposed to be used as an excuse to write 'crappy code' now and 'better code' in the future. Scrum Teams should always strive to deliver code that is maintainable, (automatically) tested, and well-written. This strategy is concerned with functional optimization. In any case, a Product Owner can more easily prioritize these items. Perhaps it is sufficient for now to let visitors search by address (and a radius), while more complex functionality (auto-completion, GPS) is implemented in the future.

Strategy 10: Breaking Down By Browser Compatibility

In a variation of strategy #4, PBI's for web-applications often need to work on a wide variety of browser platforms. Modern browsers tend to be more compliant to standards, and easier to develop for, while older browsers often require hacks and customizations to get everything to work properly.

Take this story:

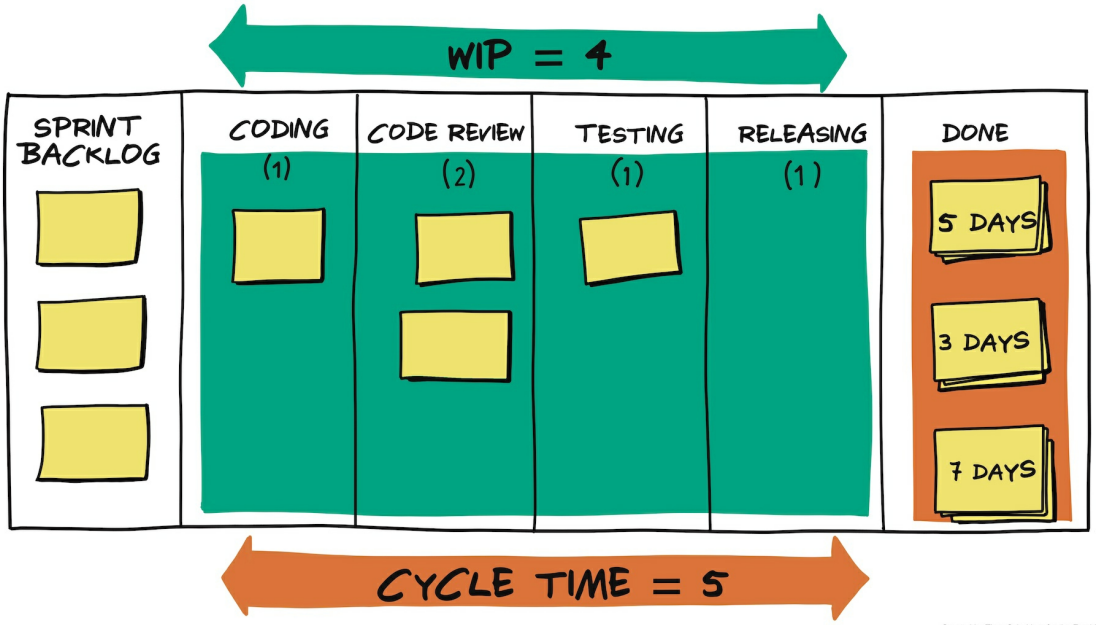
As a customer I can view product details, so I know if I want to buy it

By considering the browser versions, we can break down the work into smaller items:

- As a customer, I can view product details in a standards-compliant, modern browser, so I know if I want to buy it;
- As a customer, I can view product details in IE7, so I know if I want to buy it;
- As a customer, I can view product details in a text-browser, so I know if I want to buy it;

Although it would certainly be preferable to deliver this functionality with full support for all browser-versions, there are certainly scenarios where this break-down is feasible. Perhaps the vast majority of the visitors use modern browsers, requiring less effort to support old browsers (other than a warning). Or maybe the application is run on an internal network where all users use Internet Explorer 7. Again, this allows the Product Owner to prioritize, and helps to team to spend time and effort on the most important functionality first.

Tip: Small PBI's Optimize Flow And Minimize Work-In-Progress



Other Strategies

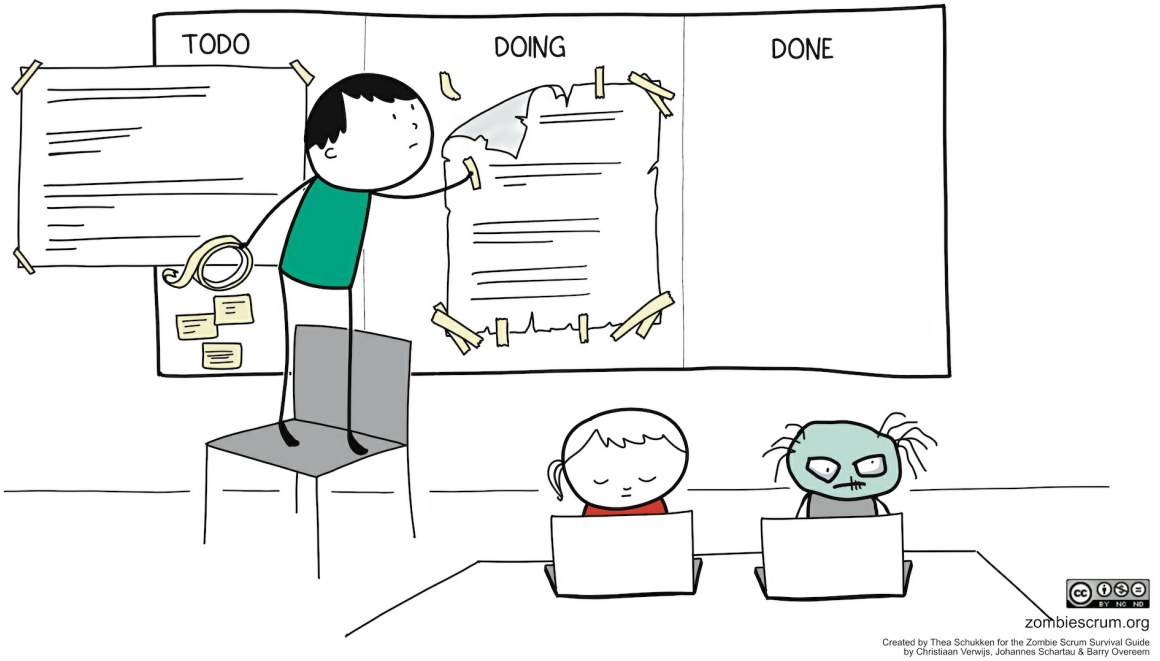
There are many other strategies that are helpful when breaking down large PBI's:

- Break down items based on identified acceptance criteria. This may seem very obvious, but it's often the easiest and most natural way to break down a story. Mapping out acceptance criteria for a PBI requires similar strategies like the ones described in this post;
- Break down items based on the parts that are hard to implement and the parts that are easier. It may be difficult to set up a piece of functionality in a heavily designed UI, but getting it to work with a simple UI may be easy and sufficient for now. Again, it's all about being pragmatic and delivering business value;
- Break down items based on external dependencies. Sometimes functionality is dependent on external factors, such as implementing a consumer for a remote web service (e.g. for electronic payment or connecting to Twitter). This may be difficult, but may not have the highest priority. Or the dependencies can be mocked for the time being;
- Break down items based on usability requirements. This includes functionality for paging through a list of records, making a website readable for blind people or people with colorblindness, or implementing breadcrumbs;
- Break down items based on SEO requirements, such as setting up dedicated landing pages for specific keywords, setting up goals for Google Analytics, or setting up XML sitemaps;

When Is A Product Backlog Item Small Enough?

There is no clear-cut rule on how small a PBI should ideally be. This greatly depends on the length of the sprints, the nature of the application, and the size and experience of the team. It often takes a Scrum Team several sprints to figure out the sweet spot.

Our own experience is that teams should strive to add at least between 5 stories to a one-week sprint (one per day). They don't have to be of equal size, but they shouldn't be too big on their own. When teams use Story Point estimation, they often agree on a maximum size for a story to being taken into a sprint. So a story will only be taken into a sprint if it's less than (say) 8 points. This also gives a clear goal to the refinement process of upcoming work.



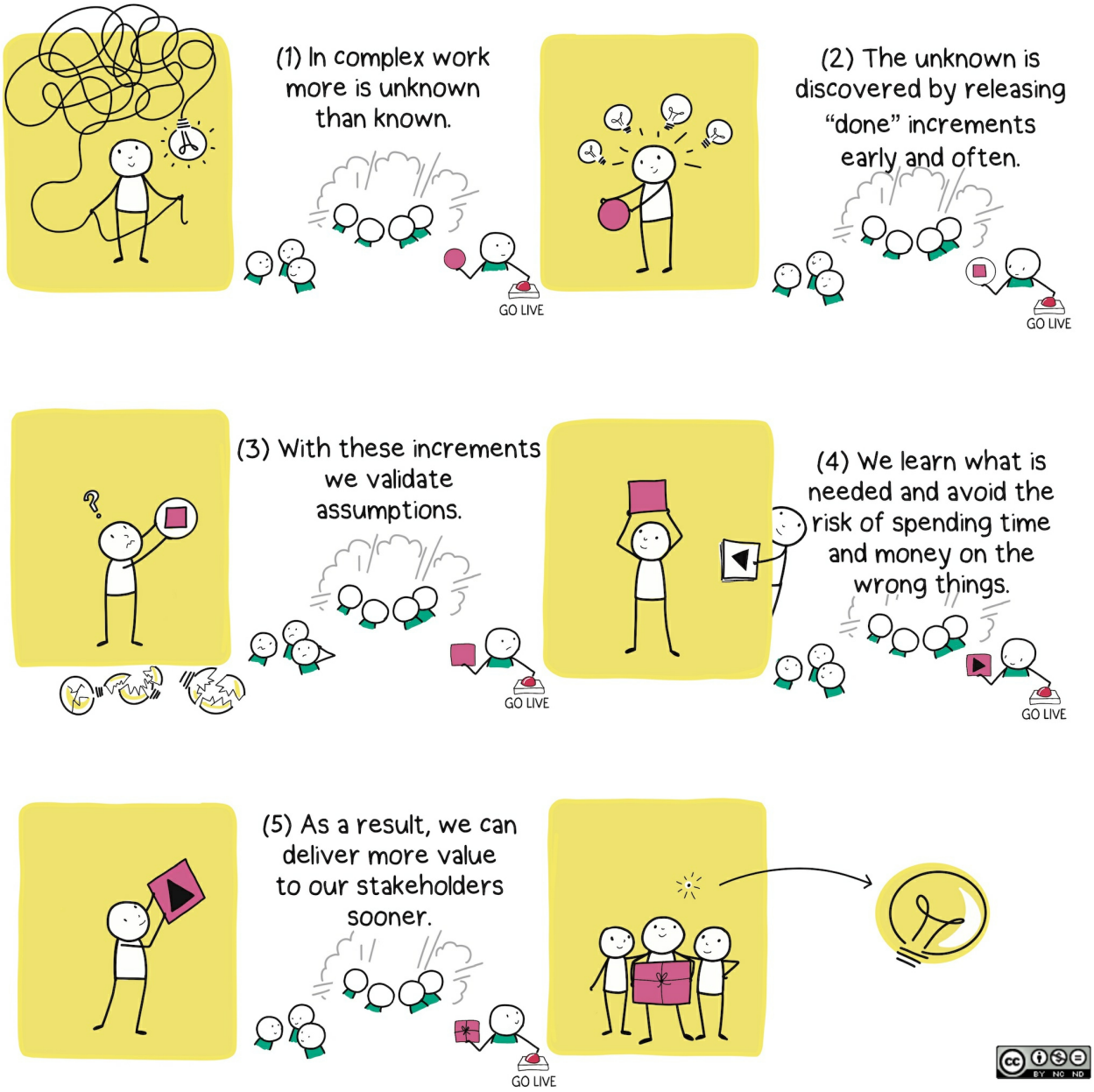
On Potential Concerns From The Scrum Team

There are some concerns that we often hear from Scrum teams when they're trying to break down PBI's. The first is that teams are worried about the reduced business value of smaller items. Of course, smaller items will have reduced business value compared to the larger item. But the primary purpose of breaking down functionality is to reduce risk, increase flow and increase the amount of working functionality that can be reviewed at the end of every sprint. The alternative is to keep working with very large chunks of functionality, with all the aforementioned consequences and risks.

Another concern is that breaking down functionality results in more work. For a team, it's often easier and faster to keep working on a piece of functionality until it's completely done. Revisiting bits and pieces throughout the sprints feel wasteful. Although this may be true to some extent, there is good reason to still do this. In Scrum, we implement a process that is designed to continuously review and test our work and adjust according to the feedback we get. So, the functionality that you may have in mind now may be very different from what will actually be implemented when we use Scrum. Although it may be tempting to keep working on a piece of functionality, there is a good chance that you are wasting your time on functionality that will be changed further down the road (based on feedback from users, stakeholders, etc).

A final concern is that many teams do not immediately 'get' how to break down functionality. It is not uncommon to run into some resistance as a result. This is understandable; trying out new things is difficult because it makes people feel vulnerable. The best way to deal with this is to persist and gently coach the team by helping them break down their PBI's.

Tip: Small PBI's Help Validate Assumptions Quickly, Manage Risk, And Deliver Value Sooner

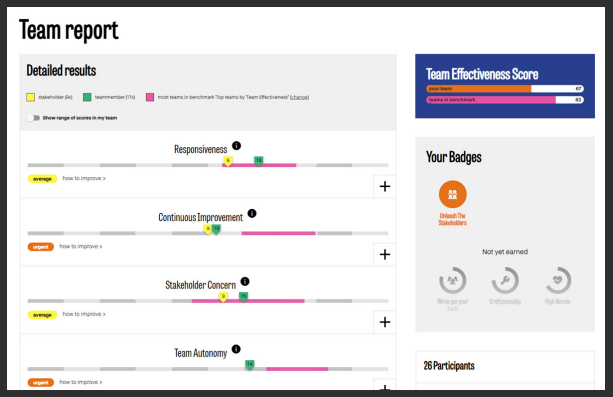


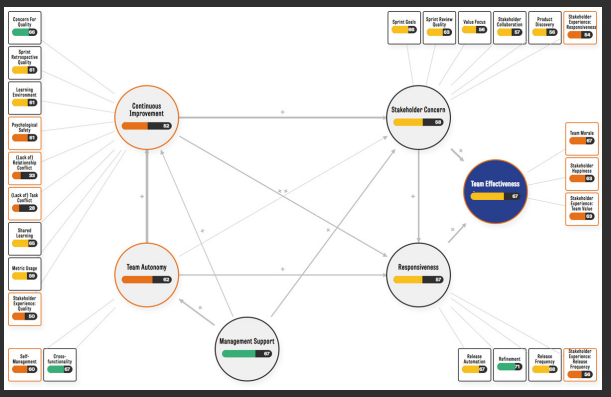
Resources & Material

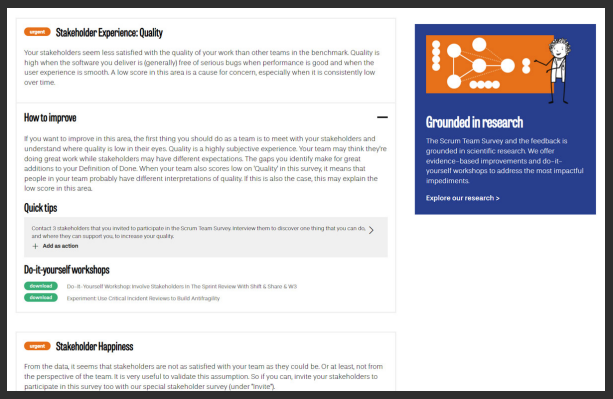
- Download a free high-resolution version of the poster in our webshop
- We also offer a physical deck of 50 Powerful Questions that includes these 10 strategies.
- Read our whitepaper where we explain the Scrum framework, and its principles and values, in our own words
- Check this DIY workshop: Narrow Down A Feature To Its Essentials In Order To Ship Faster
- Download this DIY workshop: Refine Tough Or Unclear Product Backlog Items With Stakeholders
- Splitting User Stories by Richard Lawrence.
- Five simple but powerful ways to split user stories by Mike Cohn

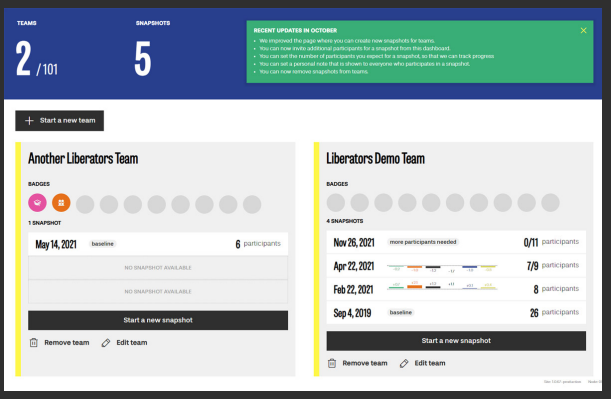
Make Your Scrum Team More Effective

Want to make your Scrum team more effective? We created the Scrum Team Survey to help. Invite members and stakeholders to a detailed questionnaire and diagnose the results together. You receive tons of actionable feedback based on your results, including dozens of quick tips and useful do-it-yourself workshops. You can use the Scrum Team Survey for free with your team. No account or e-mail address is required. You can also subscribe for more advanced features.









	Essential	Liberator
Invite team members & stakeholders	unlimited	unlimited
Quick Tips for your team per topic	3	unlimited
Recommendations to your team	5	unlimited
Compare your team against different benchmarks		✓
Analyze the range of scores in your team		✓
Receive our personal guidance on how to get started		✓
See where your team improves over time with Team Trends		✓
Support our academic research to help more teams		✓
Pricing per team per month	**€ 0**	**€ 10**

https://scrumteamsurvey.org **info@scrumteamsurvey.org**

Unleashing Teams All Over The World

We are The Liberators – Barry Overeem and Christiaan Verwijs. Our mission is to create data-driven products to unleash the superpowers of teams all over the world. We do this together with a growing community of patrons.

Awesome content for awesome teams

We unleash teams with our blogposts, our podcast, our newsletter, our videos, and our frequent meetups. While we offer most of this for free, we also have plenty of premium content in our webshop for you to explore.

Supported by the community

We are super proud that The Liberators is almost entirely funded by the community. If you appreciate our work too, you can already support us for 12 dollars/year by becoming a patron. In return, you gain free access to premium content, we share our work-in-progress and involve you in creating more awesome content.

Thank you for respecting our work!

We work hard to create high-quality content that puts you in a position to unleash your team. We're sure you appreciate that a lot of our time and money goes into this. At the same time, content like this is our main source of income. It's what pays our bills :)

If you purchased this content, we ask only that you treat our work with respect and don't share it with people outside your team. If you stumbled on it elsewhere without paying for it, and it offers you value, would you consider supporting us too? You can also check out our other offerings.